claude-windows / a9fd4426-f234-4e4a-bfb4-1f2c8255c661

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\a9fd4426-f234-4e4a-bfb4-1f2c8255c661\subagents\workflows\wf_076beba8-59c\agent-acfc77bca47ec157b.jsonl`
- SHA-256: `53c1cad4a78a77a930768cf5a59166c83e80069a277d6d708290b1204042fa26`
- Source modified: `2026-05-31T04:22:14+00:00`
- Imported at: `2026-07-05T16:48:20+00:00`
- project: `wf_076beba8-59c`
- session_id: `a9fd4426-f234-4e4a-bfb4-1f2c8255c661`

Transcript

1. user (2026-05-31T04:21:06.024Z)

Adversarial Claim Verifier (voter 2/3)

Be SKEPTICAL. Try to REFUTE this claim. ?2/3 refutations kill it.

Research question

Identify the best LOCAL (self-hosted, single NVIDIA CUDA GPU) vision / document-AI / OCR models ? and, as a clearly-flagged approval-gated fallback only, paid cloud API services ? for parsing BANK STATEMENT PDFs into structured transaction tables (date, narration/description, withdrawal/debit, deposit/credit, running balance), for a LOCAL-FIRST personal-finance tool ("muneem").

Target documents: Indian bank statements (HDFC, Axis, ICICI, AU Small Finance) ? dense, multi-column, often SEMI-RULED or borderless tables, sometimes multiple accounts per statement; both born-digital (text layer present) and the harder cases where pdfplumber's extract_tables/coordinate clustering is unreliable (e.g. Axis, whose column layout varies between statements).

HARD CONSTRAINTS (a model that violates these is unusable for us):
1. MUST run 100% LOCALLY for statement contents (highly sensitive). NO cloud APIs by default. Paid/cloud is acceptable ONLY as an explicitly-approved fallback ? still report the best cloud options but label them clearly as "cloud/sensitive ? approval-gated, not default".
2. MUST be usable from Python (transformers / vLLM / Ollama / ONNX Runtime / native lib).
3. License MUST be permissive & commercially safe for BOTH the CODE and the MODEL WEIGHTS: MIT, BSD, Apache-2.0, ISC, HPND. AVOID and explicitly FLAG anything GPL/LGPL/AGPL, CC-BY-NC / non-commercial, research-only, or custom "community"/acceptable-use licenses with restrictions. This is critical ? many document-AI models and their training datasets are non-commercial (e.g. LayoutLMv3 weights are CC-BY-NC; Surya and Marker use GPL-or-commercial dual licensing; several VLM weights ship custom restrictive licenses). Verify the WEIGHTS license, not just the repo license.
4. Hardware: a single consumer/prosumer NVIDIA GPU. Give VRAM tiers (~8-12 GB consumer, ~16-24 GB) and which models/quantizations fit each.

Evaluate and compare these LOCAL approaches ? for each give: code license, MODEL-WEIGHTS license (state explicitly), latest release / maintenance status as of 2025-2026, VRAM need + quantization options, Python integration path, and reliability specifically on DENSE FINANCIAL TABLES / multi-column statements:
- Open vision-language models (VLMs) for document/table understanding: Qwen2-VL and Qwen2.5-VL (note per-size license ? which sizes are Apache-2.0 vs restricted), MiniCPM-V 2.6, InternVL2 / 2.5, Microsoft Phi-3.5-vision, Mistral Pixtral, Meta Llama-3.2-Vision (flag the Llama Community License restrictions), Moondream2, GOT-OCR2.0, olmOCR and other OCR-specialized VLMs.
- OCR-specialist / document-AI pipelines: PaddleOCR + PP-Structure (table recognition), docTR, EasyOCR, Tesseract (already used), Microsoft Table Transformer / TATR (check code AND PubTables-1M dataset/weights license), Donut, Surya (FLAG license), Marker (FLAG license),

MinerU / PDF-Extract-Kit, RapidOCR.
- Table-structure recognition specifically (cell/grid reconstruction for borderless tables): TATR, PP-Structure, UniTable, etc.

Also assess: for SEMI-RULED / borderless bank tables, is a prompt-driven VLM (page image -> JSON transactions) more reliable than a classic OCR + table-structure pipeline? What are the accuracy/hallucination risks of VLMs on financial NUMBERS (transposed digits, dropped/duplicated rows), and how should output be VERIFIED? (muneem already machine-checks parses by balance reconciliation ? opening + sum(deposits) - sum(withdrawals) == closing, plus per-row running-balance walks and cross-checks against the bank's printed summary ? so any model's output can be automatically validated, which should inform the recommendation.)

Finally, briefly cover the best PAID/CLOUD fallbacks (clearly flagged approval-gated for sensitive data): AWS Textract (AnalyzeExpense / Tables), Google Document AI, Azure AI Document Intelligence, Reducto, LlamaParse, Anthropic/OpenAI vision ? note any self-hosting option and data-handling terms.

Deliver a RANKED shortlist of the most promising LOCAL options for muneem (NVIDIA GPU, Python, commercial-license-safe), each with license (code + weights), VRAM tier, maturity, strengths/weaknesses for dense bank-statement tables, and a concrete recommendation: which local model(s) to prototype for muneem's statement parsing and how they'd combine with the existing deterministic parsers + balance-reconciliation verification (e.g. VLM as a fallback/cross-check when deterministic parsing fails to reconcile). Cite sources and call out any license uncertainty explicitly.

Claim under review

"olmOCR 2 is built on Qwen2.5-VL-7B-Instruct, a 7B-parameter vision-language model, fine-tuned then RL-trained; this anchors its VRAM/hardware footprint to the 7B Qwen2.5-VL tier (roughly 16-24 GB GPU class at FP16, less when quantized)."

Source: https://arxiv.org/html/2510.19817v1 (primary)
Supporting quote: "We start with a Qwen2.5-VL-7B-Instruct model that has been fine-tuned on olmOCR-mix-1025."

Checklist

1. Is the claim actually supported by the quote, or is it an overreach/misread?
2. WebSearch for contradicting evidence ? does any credible source dispute or heavily qualify this?
3. Is the source quality sufficient for the claim's strength? (extraordinary claims need primary sources)
4. Is the claim outdated? (check dates ? old claims about fast-moving fields are suspect)
5. Is this a marketing claim / press release / cherry-picked benchmark / forum speculation?

refuted=true if: unsupported by quote / contradicted / low-quality source for strong claim / outdated / marketing fluff.
refuted=false ONLY if: claim is well-supported, current, and source quality matches claim strength.
Default to refuted=true if uncertain.

Structured output only. Evidence MUST be specific.

2. assistant (2026-05-31T04:21:09.364Z)

I'll verify this claim about olmOCR 2's architecture and VRAM footprint. Let me start by examining the primary source and searching for corroborating/contradicting evidence.

3. assistant (2026-05-31T04:21:12.968Z)

Let me read the primary source and search for corroborating evidence in parallel.

4. user (2026-05-31T04:21:19.110Z)

olmOCR 2 Base Model and Training

Yes, olmOCR 2 is built on Qwen2.5-VL-7B-Instruct.

Base Model Architecture

The document states: "olmOCR 2 is powered by olmOCR-2-7B-1025, a specialized, 7B vision language model (VLM) trained using reinforcement learning with verifiable rewards (RLVR)."

The training process begins with: "We start with a Qwen2.5-VL-7B-Instruct model that has been fine-tuned on olmOCR-mix-1025."

Training Approach

The document describes a two-stage training pipeline:
1. Supervised fine-tuning on olmOCR-mix-1025 dataset
2. Reinforcement learning training using "Group Relative Policy Optimization (GRPO)" with binary unit test rewards on synthetic data (olmOCR2-synthmix-1025)

They further note: "we train six models with different random seeds, and soup their weights at the end."

Hardware Requirements

The document provides limited hardware details, mentioning only: "We train for one epoch on olmOCR2-synthmix-1025 using an 8xH100 GPU node."

No specific VRAM requirements or memory footprint information is disclosed in the provided content.

5. user (2026-05-31T04:21:22.856Z)

Web search results for query: "olmOCR 2 Qwen2.5-VL-7B base model architecture RL training"

Links: [{"title":"Papers Explained 480: olmOCR 2 | by Ritvik Rastogi | Medium","url":"https://ritvik19.medium.com/papers-explained-480-olmocr-2-33fbf89757eb"},{"title":"richardyoung/olmOCR-2-7B-1025-MLX-6bit · Hugging Face","url":"https://huggingface.co/richardyoung/olmOCR-2-7B-1025-MLX-6bit"},{"title":"olmOCR 2: Advanced Open-Source OCR","url":"https://www.emergentmind.com/topics/olmocr-2"},{"title":"olmOCR 2: Unit test rewards for document OCR | Ai2","url":"https://allenai.org/blog/olmocr-2"},{"title":"allenai/olmOCR-2-7B-1025-FP8 · Hugging Face","url":"https://huggingface.co/allenai/olmOCR-2-7B-1025-FP8"},{"title":"Model Request: Add support for olmOCR-2 (based on Qwen2.5-VL-7B-Instruct) · Issue #373 · EricLBuehler/candle-vllm","url":"https://github.com/EricLBuehler/candle-vllm/issues/373"},{"title":"allenai/olmOCR-2-7B-1025 · Hugging Face","url":"https://huggingface.co/allenai/olmOCR-2-7B-1025"},{"title":"2 OLMo 2 Furious","url":"https://arxiv.org/pdf/2501.00656"},{"title":"olmOCR-2-7B-1025 OCR Model","url":"https://www.emergentmind.com/topics/olmocr-2-7b-1025"}]

Based on the search results, here's what I found about olmOCR 2's architecture and RL training:

Base Model Architecture

olmOCR 2 is built on Qwen2.5-VL-7B and fine-tuned on olmOCR-mix-1025, a dataset of 270,000 PDF pages with diverse properties?academic papers, historical scans, legal documents, brochures, and more. The underlying VLM leverages the vision transformer backbone, adapted for both scanned and born-digital page images, supporting the advanced handling of tables, equations, and complex layouts.

RL Training Approach

olmOCR 2 is a specialized, 7B vision language model (VLM) trained using reinforcement learning with verifiable rewards (RLVR), where the rewards are a diverse set of binary unit tests. More specifically, training is governed by a reinforcement learning protocol?Group Relative Policy

Optimization (GRPO)?using binary unit tests as the reward function.

Training Pipeline

The best model is the result of: A single epoch of SFT training on olmOCR-mix-1025, A single epoch of RL training over synthetic data olmOCR2-synthmix-1025, Repeating the RL training for six random seeds and averaging (or "souping") the checkpoints.

Reward System

Each completion gets scored using the standard olmOCR-Bench scoring rules, where each test case is either a pass or fail, and the reward is the fraction from 0.0 to 1.0 of passing test cases. Besides the unit test above, two additional rewards are included to ensure correct output format: a binary reward for whether the model completion ends with the EOS token, and a reward between 0 and 1 to ensure that the model outputs document metadata at the top of its response.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

6. assistant (2026-05-31T04:21:26.963Z)

The primary source strongly confirms the architectural claims. Now let me verify the specific VRAM/hardware footprint claim (16-24 GB at FP16, less when quantized) by checking the official model cards and inference requirements.

7. user (2026-05-31T04:21:32.922Z)

olmOCR-2-7B-1025 Model Details

Base Model

Base Model: Qwen2.5-VL-7B-Instruct

From the page: *"This is a release of the olmOCR model that's fine tuned from **Qwen2.5-VL-7B-Instruct** using the olmOCR-mix-1025 dataset."*

The model tree section also confirms: "Base model: [Qwen/Qwen2.5-VL-7B-Instruct](/Qwen/Qwen2.5-VL-7B-Instruct)"

GPU/VRAM Requirements

The page does not specify minimum GPU memory requirements for inference.

Quantized Versions

Yes, quantized versions are available:

1. **FP8 Version** - Recommended for practical use:
   - [olmOCR-2-7B-1025-FP8](https://huggingface.co/allenai/olmOCR-2-7B-1025-FP8)
   - Quote: *"Full BF16 version of olmOCR-2-7B-1025-FP8. We recommend using the FP8 version for all practical purposes except further fine tuning."*

2. **Additional Quantizations** - 11 quantized models available for:
   - llama.cpp
   - LM Studio
   - Jan
   - Ollama

The current model shown is **BF16** format with **8B parameters**. The FP8 version would provide reduced memory requirements while maintaining similar performance (FP8 scores: 82.4 ± 1.1 vs BF16: 82.3 ± 1.1).

8. user (2026-05-31T04:21:34.450Z)

Web search results for query: "olmOCR-2-7B-1025 GPU VRAM requirement inference how much memory run"

Links: [{"title":"GitHub - allenai/olmocr: Toolkit for linearizing PDFs for LLM datasets/training · GitHub","url":"https://github.com/allenai/olmocr"},{"title":"bartowski/allenai_olmOCR-2-7B-1025-GGUF · Hugging Face","url":"https://huggingface.co/bartowski/allenai_olmOCR-2-7B-1025-GGUF"},{"title":"bartowski/allenai_olmOCR-2-7B-1025-GGUF - MyGGUF Model Details","url":"https://mygguf.com/models/bartowski_allenai_olmOCR-2-7B-1025-GGUF"},{"title":"allenai/olmOCR-2-7B-1025-FP8 - Fast, Reliable, and Scalable Inference on FriendliAI","url":"https://friendli.ai/model/allenai/olmOCR-2-7B-1025-FP8"},{"title":"allenai/olmOCR-2-7B-1025 · Hugging Face","url":"https://huggingface.co/allenai/olmOCR-2-7B-1025"},{"title":"allenai/olmOCR-2-7B-1025 - Fast, Reliable, and Scalable Inference on FriendliAI","url":"https://friendli.ai/models/allenai/olmOCR-2-7B-1025"},{"title":"Can You Run This LLM? VRAM Calculator (Nvidia GPU and Apple Silicon)","url":"https://apxml.com/tools/vram-calculator"},{"title":"Deploying olmOCR 2: High-throughput OCR on Spheron Decentralized GPU","url":"https://blog.spheron.network/deploying-olmocr-2-high-throughput-ocr-on-spheron-decentralized-gpu"},{"title":"Ollama VRAM Requirements: Complete 2026 Guide to GPU Memory for Local LLMs | LocalLLM.in","url":"https://localllm.in/blog/ollama-vram-requirements-for-local-llms"},{"title":"FP8 model extremely slow on 12GB VRAM GPU due to FP16 upcast and hardcoded VRAM check · Issue #415 · allenai/olmocr","url":"https://github.com/allenai/olmocr/issues/415"}]

Based on the search results, here's what I found regarding GPU VRAM requirements for running olmOCR-2-7B-1025:

VRAM Requirements

Consumer GPUs with 8 to 24 GB VRAM can run these models with suitable batch sizes, while large-scale production setups use A100 or H100 GPUs with 40?80 GB VRAM, enabling higher batch throughput.

FP8 Variant Performance

The FP8-quantized version (olmOCR-2-7B-1025-FP8) has some interesting considerations. The FP8 quantized model runs at approximately 0.5 tokens/second (~20 minutes per page) on a 12GB VRAM GPU. However, this is due to a technical limitation: The transformers library does not support native FP8 inference, and despite the model being stored in FP8, weights are upcast to FP16 during computation, increasing memory usage from ~7GB to ~20GB, exceeding dedicated VRAM and spilling into shared system memory.

General Quantization Guidelines

For running quantized versions of the model, aim for a quant with a file size 1-2GB smaller than your GPU's total VRAM. This means for optimal performance, you'd want at least 10-14GB of VRAM for comfortable operation.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

9. assistant (2026-05-31T04:21:37.570Z)

I now have strong corroboration. Let me cross-check the official olmocr GitHub repo for the documented hardware requirement, since that's the authoritative deployment source and will pin down the exact VRAM floor the project recommends.

10. user (2026-05-31T04:21:42.473Z)

olmOCR GPU Requirements

Based on the documentation, here are the hardware specifications:

Minimum VRAM: The system requires "at least 12 GB of GPU RAM" for local inference.

Recommended GPUs: The toolkit has been "tested on RTX 4090, L40S, A100, H100" according to the installation guide.

Additional Storage: You'll need "30GB of free disk space" for the full installation.

Regarding the base model: The webpage does not specify that olmOCR uses Qwen2.5-VL-7B-Instruct as its foundation. The documentation mentions the system is "based on a 7B parameter VLM" and references training code for "Qwen2.5-VL," but does not explicitly confirm this as the base model for the released olmOCR versions. The default model referenced is "allenai/olmOCR-2-7B-1025-FP8."

11. assistant (2026-05-31T04:21:56.229Z)

I have sufficient evidence to evaluate this claim thoroughly. Let me assess each component against the checklist.

Claim breakdown and verification:

1. **"Built on Qwen2.5-VL-7B-Instruct, a 7B-parameter VLM"** ? CONFIRMED by primary source (arXiv 2510.19817v1) and the official HF model card: "fine tuned from Qwen2.5-VL-7B-Instruct" and "model tree: Base model: Qwen/Qwen2.5-VL-7B-Instruct." The supporting quote ("We start with a Qwen2.5-VL-7B-Instruct model...") directly supports this.

2. **"Fine-tuned then RL-trained (GRPO/RLVR)"** ? CONFIRMED. Paper: "trained using reinforcement learning with verifiable rewards (RLVR)" + SFT on olmOCR-mix-1025 then GRPO RL. Accurate.

3. **"Anchors VRAM to the 7B tier"** ? Architecturally sound. A Qwen2.5-VL-7B derivative inherits the 7B parameter count.

4. **"roughly 16-24 GB GPU class at FP16, less when quantized"** ? This is the only numerically-specific sub-claim and the one most worth scrutinizing:
   - FP16/BF16 weights for 7B ? ~14-16 GB, plus the vision encoder + KV cache + activations on high-res document images pushes real usage into the ~16-24 GB band. Corroborated independently: the GitHub issue documents FP16 computation using "~20GB" of memory.
   - "less when quantized" ? CONFIRMED: official FP8 variant exists (~7 GB weights), plus GGUF/Ollama quants.
   - Note: the official repo states a **12 GB minimum**, which is *below* the claim's 16-24 GB band. But the claim says "roughly 16-24 GB GPU *class* at FP16" (i.e., comfortable FP16 operation), and explicitly adds "less when quantized" ? the 12 GB floor is for quantized/constrained operation (and the issue shows 12 GB FP8 is painfully slow due to FP16 upcast). So the 12 GB figure does not contradict the claim; it actually supports the "less when quantized" portion. The 16-24 GB FP16 characterization is a reasonable, well-grounded estimate, hedged appropriately with "roughly."

The claim is well-supported by primary sources, current (Oct 2025 release, checked May 2026), correctly hedged ("roughly," "class," "less when quantized"), and not marketing fluff. Nothing contradicts it; the one number slightly below the band (12 GB) is explained by the claim's own "less when quantized" qualifier.

12. user (2026-05-31T04:22:05.834Z)

Structured output provided successfully

13. assistant (2026-05-31T04:22:14.256Z)

I've completed the adversarial verification of this claim and returned my finding via StructuredOutput: **refuted=false** (claim upheld, high confidence).

Summary of my attempt to refute and why it failed:

- **Architecture (Qwen2.5-VL-7B-Instruct, 7B params, SFT+RL)** ? directly confirmed by both the primary arXiv source and the independent official HuggingFace model card. No overreach; the

supporting quote accurately backs it.
- **VRAM footprint (16-24 GB at FP16, less when quantized)** ? the one numeric sub-claim I scrutinized hardest. It is well-grounded (7B BF16 ?14-16 GB + vision encoder + KV cache lands in-band; corroborated by olmocr issue #415 reporting ~20 GB FP16 usage). The official FP8 variant confirms "less when quantized."
- **Strongest counter-evidence found:** the olmocr GitHub README's stated **12 GB minimum**, which sits below the claimed band. But this does not refute the claim ? the claim explicitly hedges with "roughly," "GPU class," and "less when quantized," and issue #415 shows 12 GB only works (very slowly) under quantization, not at FP16. So the counter actually reinforces the claim rather than contradicting it.

The claim is well-supported, current (Oct 2025 release, valid as of May 2026), properly hedged, and not marketing fluff. One refutation attempt at most, well short of the 2/3 needed to kill it.